

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

*Факультет Информационных технологий
Кафедра Информатики и информационных технологий*

направление подготовки

09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 9

Дисциплина: BackEnd-разработка

Тема: Конфигурация веб-приложение на основе ASP.NET Core

Выполнил: студент группы 241 - 333

Конев Максим Дмитриевич
(Фамилия И.О.)

Дата, подпись _____
(Дата) (Подпись)

Проверил: _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____
(Дата) (Подпись)

Замечания: _____

Москва

2026

Цель работы

Освоить процесс конфигурирования веб-приложений на платформе ASP.NET Core для эффективного управления параметрами приложения.

Примечание

Данная лабораторная работа является расширенной версией предыдущей лабораторной работы, в следствие чего могут повторяться некоторые её отдельные части.

Используемые технологии и инструментальные средства

В работе применялись:

- ASP.NET Core Web API — разработка HTTP API
- C# — язык реализации
- Visual Studio — среда разработки
- Swagger (OpenAPI) — тестирование API
- IConfiguration — доступ к конфигурации
- IOptions — привязка конфигурации к классу
- DataAnnotations — валидация входных данных

Ход выполнения работы

1. Создание проекта

Был создан проект типа ASP.NET Core Web API. После создания проект содержит:

- точку входа Program.cs
- папку Controllers
- файл конфигурации appsettings.json

Основная задача — реализовать API для управления студентами с возможностью управления логикой через конфигурацию.

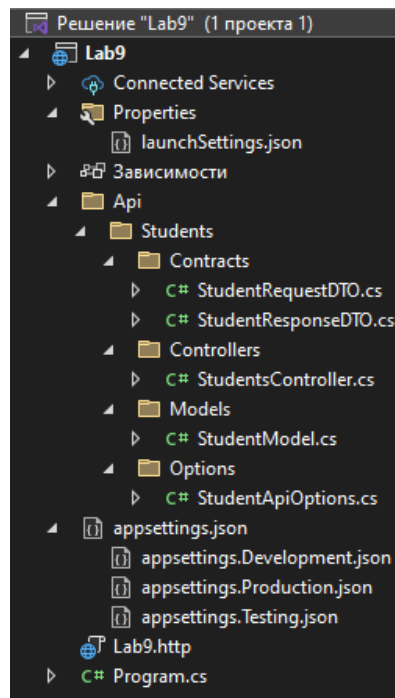


Рисунок 1. Структура проекта

2. Организация конфигурации приложения

В проект добавлены несколько конфигурационных файлов:

1. Базовый файл — appsettings.json

```
"StudentApiSettings": {
  "MaxCountStudents": 10,
  "AllowAdd": true,
  "AllowDelete": true,
  "AllowUpdate": true,
  "Code": "FROM <ENV>"
},
```

Рисунок 2. appsettings.json

Назначение:

- задаёт значения по умолчанию
- используется во всех средах, если не переопределён

2. Конфигурация разработки — appsettings.Development.json

```
"StudentApiSettings": {  
  "MaxCountStudents": 5,  
  "AllowAdd": false,  
  "AllowDelete": false,  
  "AllowUpdate": false  
},
```

Рисунок 3. appsettings.Development.json

Особенности:

- сильно ограничивает функциональность
- используется для тестирования ограничений

3. Конфигурация Production

```
"StudentApiSettings": {  
  ...  
},
```

Рисунок 4. appsettings.Production.json

Особенность:

- значения берутся из:
 - базового файла
 - переменных окружения

4. Конфигурация Testing

```
"StudentApiSettings": {  
  ...  
  "MaxCountStudents": 10000  
},
```

Рисунок 5. appsettings.Testing.json

Назначение:

- тестирование высокой нагрузки

3. Использование переменных окружения

В файле launchSettings.json:

```
"ASPNETCORE_ENVIRONMENT": "Production",  
"StudentApiSettings__Code": "12345"
```

Рисунок 6. launchSettings.json

Объяснение:

1. ASPNETCORE_ENVIRONMENT = Production
 - активирует файл appsettings.Production.json
2. StudentApiSettings__Code
 - __ означает вложенность: StudentApiSettings.Code
3. Приоритет конфигурации:
 - a. appsettings.json
 - b. appsettings.Environment.json
 - c. Environment Variables (самый высокий приоритет)

4. Настройка загрузки конфигурации

В Program.cs:

```
builder.Configuration  
    .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)  
    .AddJsonFile($"appsettings.{builder.Environment.EnvironmentName}.json", optional: false, reloadOnChange: true)  
    .AddEnvironmentVariables();
```

Рисунок 7. Загрузка конфигурации

Разбор:

- AddJsonFile — подключает JSON
- reloadOnChange — обновление без перезапуска
- AddEnvironmentVariables — подключает ENV

5. Реализация Options Pattern

Создан класс:

```

public class StudentApiOptions
{
    Ссылка: 1
    public int MaxCountStudents { get; init; }
    Ссылка: 1
    public bool AllowAdd { get; init; }
    Ссылка: 1
    public bool AllowDelete { get; init; }
    Ссылка: 1
    public bool AllowUpdate { get; init; }
    Ссылка: 1
    public string Code { get; init; }
}

```

Рисунок 8. Options pattern

Регистрация:

```

builder.Services.Configure<StudentApiOptions>(builder.Configuration.GetRequiredSection("StudentApiSettings"));

```

Рисунок 9. Регистрация конфига

Суть:

- конфиг автоматически мапится в объект
- далее используется через DI

6. Реализация контроллера

Контроллер:

```

public class StudentsController(IOptions<StudentApiOptions> options) : ControllerBase

```

Рисунок 10. Реализация контроллера

Используем `IOptions<StudentApiOptions>` чтобы отделить код от источника данных и получить актуальную конфигурацию.

7. Реализация логики API

Получение конфигурации:

```
[HttpGet("/cfg")]
Ссылка: 0
public ActionResult GetConfig()
{
    return Ok(options.Value.Code);
}
```

Рисунок 11. GET-метод для получения конфига

Также в методы добавления, обновления и удаления студентов добавлены дополнительные проверки на разрешение этих операций:

```
if (_students.Count >= options.Value.MaxCountStudents)
{
    return BadRequest("Достигнуто максимальное количество студентов.");
}

if (!options.Value.AllowAdd)
{
    return BadRequest("Данная опция недоступна.");
}
```

Рисунок 12. Проверка на количество студентов и на доступ к операции добавления

```
if (!options.Value.AllowUpdate)
{
    return BadRequest("Данная опция недоступна.");
}
```

Рисунок 13. Проверка на доступ к операции обновления

```
if (!options.Value.AllowDelete)
{
    return BadRequest("Данная опция недоступна.");
}
```

Рисунок 14. Проверка на доступ к операции удаления

8. Скриншоты приложения

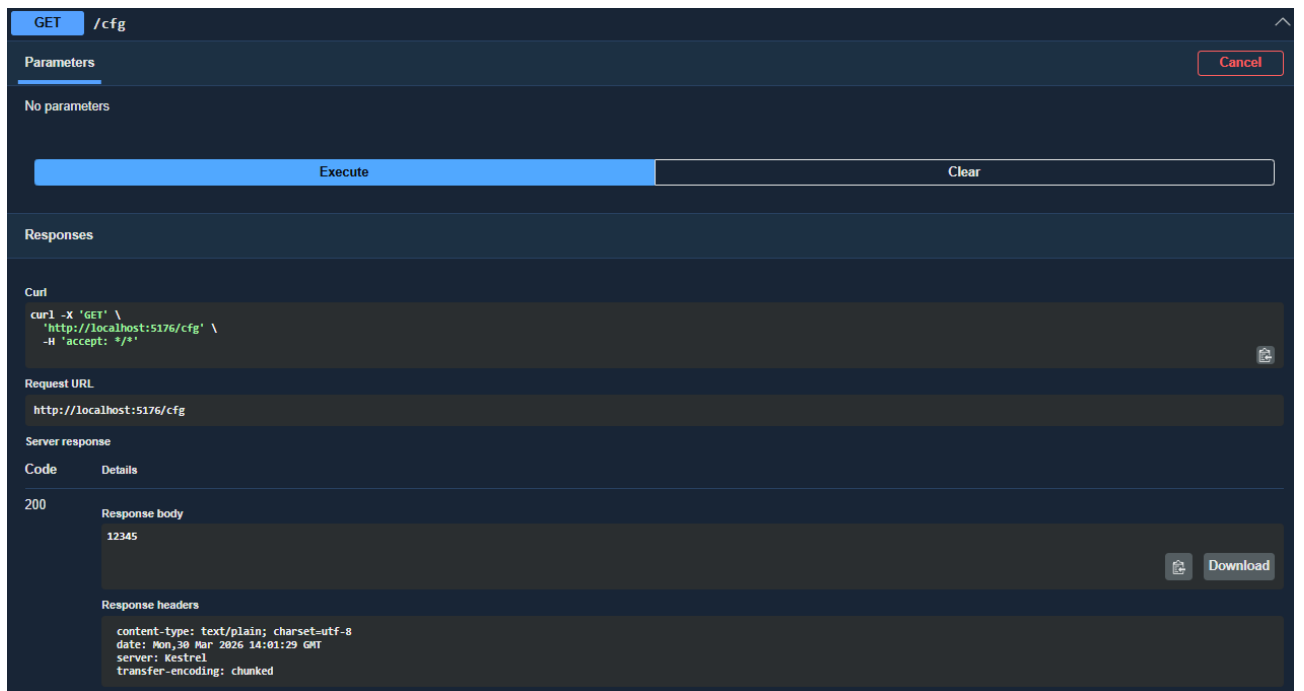


Рисунок 15. GET-запрос кода конфига и его результат

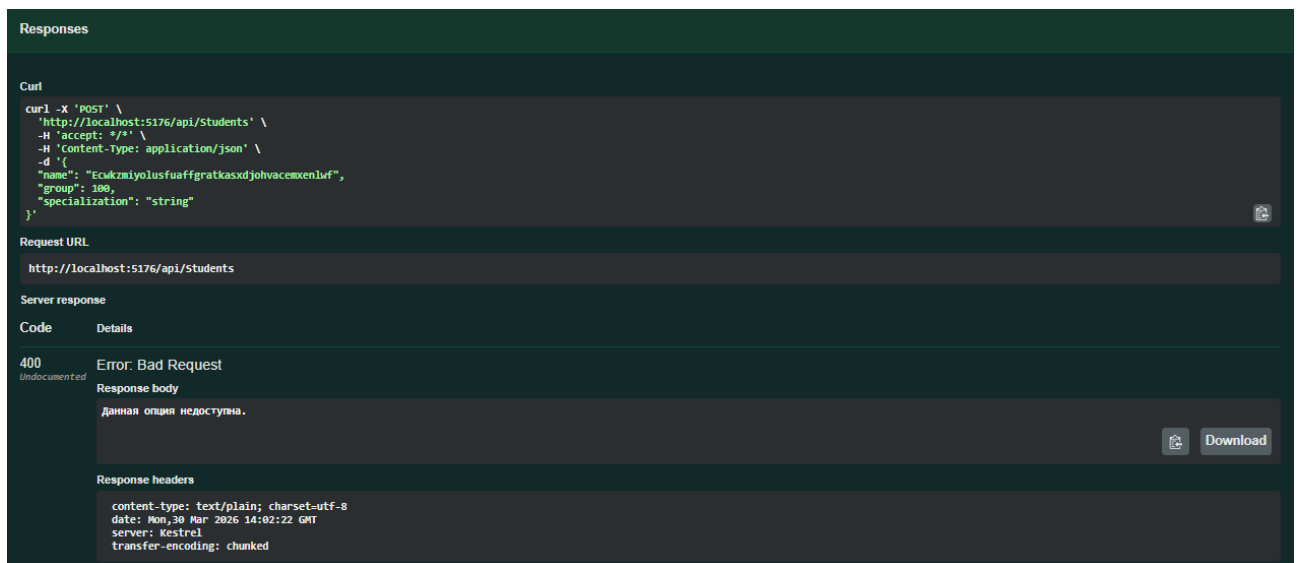


Рисунок 16. Результат POST-запроса при среде Development

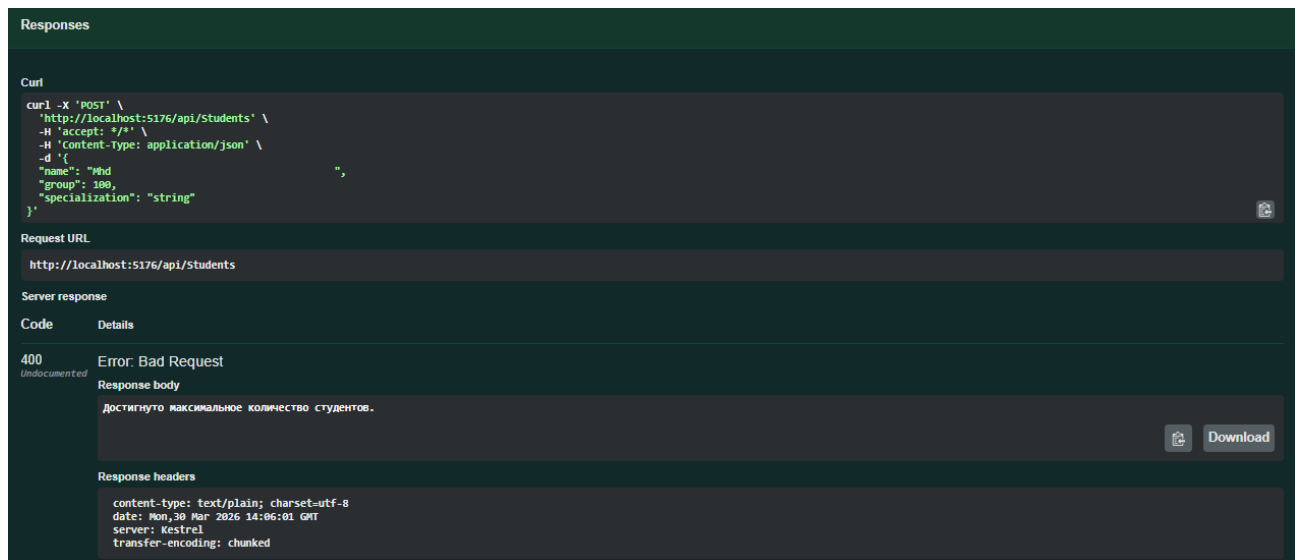


Рисунок 17. Результат 11-го POST-запроса при среде Production

9. Возможные сложности

В процессе выполнения могли возникнуть следующие сложности:

- Понимание приоритетов конфигурации
- Правильное использование __ в переменных окружения
- Работа с IOption
- Валидация DTO
- Логика ограничения через конфиг

Вывод

В ходе работы:

- освоена система конфигурации ASP.NET Core
- изучен механизм приоритетов настроек
- реализовано динамическое управление API
- применён паттерн Options
- реализована валидация входных данных

Практическая часть показала, что конфигурация позволяет управлять поведением приложения без изменения кода, что критически важно для production-среды.

Листинг кода

Листинг 1. Файл Program.cs:

```
using Lab9.Api.Students.Options;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSwaggerGen();

builder.Services.AddControllers();

builder.Configuration
    .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
    .AddJsonFile($"appsettings.{builder.Environment.EnvironmentName}.json", optional:
false, reloadOnChange: true)
    .AddEnvironmentVariables();

builder.Services.Configure<StudentApiOptions>(builder.Configuration.GetRequiredSection
("StudentApiSettings"));

var app = builder.Build();

app.UseSwagger().UseSwaggerUI();

app.MapControllers();

app.Run();
```

Листинг 2. Файл StudentRequestDTO.cs:

```
using System.ComponentModel.DataAnnotations;

namespace Lab9.Api.Students.Contracts
{
    public class StudentRequestDTO
    {
        [Required]
        [MaxLength(40)]
        [MinLength(2)]
        [RegularExpression(@"^([A-ZА-Я][a-za-я]*[ ]*)+$")]
        public required string Name { get; init; }
        [Required]
        [Range(1, 100)]
        public required int Group { get; init; }
        [Required]
        public required string Specialization { get; init; }
    }
}
```

Листинг 3. Файл StudentResponseDTO.cs:

```
namespace Lab9.Api.Students.Contracts
{
    public class StudentResponseDTO
    {
        public required int Id { get; init; }
        public required string Name { get; init; }
        public required string Specialization { get; init; }
        public required int Group { get; init; }
    }
}
```

```
}  
}
```

Листинг 4. Файл StudentsController.cs:

```
using Lab9.Api.Students.Contracts;  
using Lab9.Api.Students.Models;  
using Lab9.Api.Students.Options;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.Extensions.Options;  
  
namespace Lab9.Api.Students.Controllers  
{  
    [Route("api/[controller]")]  
    [ApiController]  
    public class StudentsController(IOptions<StudentApiOptions> options) :  
        ControllerBase  
    {  
        // GET метод для конфига  
        [HttpGet("/cfg")]  
        public ActionResult GetConfig()  
        {  
            return Ok(options.Value.Code);  
        }  
  
        // GET метод  
        [HttpGet]  
        public ActionResult<StudentResponseDTO[]> GetAllStudents()  
        {  
            var students = _students.Select(x => new StudentResponseDTO  
            {  
                Id = x.Id,  
                Name = x.Name,  
                Group = x.Group,  
                Specialization = x.Specialization  
            }).ToArray();  
  
            if (students.Any())  
            {  
                return Ok(students);  
            }  
            return NotFound("Список студентов пуст.");  
        }  
  
        // GET{id} метод  
        [HttpGet]  
        [Route("{id}")]  
        public ActionResult<StudentResponseDTO> GetStudent(int id)  
        {  
            var student = _students.FirstOrDefault(x => x.Id == id);  
            if (student == null) return NotFound("Студент не найден.");  
  
            return Ok(new StudentResponseDTO  
            {  
                Id = student.Id,  
                Name = student.Name,  
                Group = student.Group,  
                Specialization = student.Specialization  
            });  
        }  
  
        // POST метод  
        [HttpPost]  
        public ActionResult CreateStudent([FromBody] StudentRequestDTO contract)
```

```

{
    if (_students.Count >= options.Value.MaxCountStudents)
    {
        return BadRequest("Достигнуто максимальное количество студентов.");
    }

    if (!options.Value.AllowAdd)
    {
        return BadRequest("Данная опция недоступна.");
    }

    var id = _students.Count == 0 ? 1 : _students.Max(x => x.Id) + 1;

    var student = new Student
    {
        Id = id,
        Name = contract.Name,
        Group = contract.Group,
        Specialization = contract.Specialization,
    };

    _students.Add(student);

    return CreatedAtAction(nameof(GetStudent), new { id = student.Id }, new
StudentResponseDTO
    {
        Id = student.Id,
        Name = student.Name,
        Group = student.Group,
        Specialization = student.Specialization
    });
}

// PUT{id} метод
[HttpPut("{id}")]
public ActionResult UpdateStudent(int id, [FromBody] StudentRequestDTO
contract)
{
    if (!options.Value.AllowUpdate)
    {
        return BadRequest("Данная опция недоступна.");
    }

    var student = _students.FirstOrDefault(x => x.Id == id);

    if (student == null) return NotFound();

    student.Name = contract.Name;
    student.Group = contract.Group;
    student.Specialization = contract.Specialization;

    return NoContent();
}

// DELETE{id} метод
[HttpDelete]
[Route("{id}")]
public ActionResult DeleteStudent(int id)
{
    if (!options.Value.AllowDelete)
    {
        return BadRequest("Данная опция недоступна.");
    }

    var index = _students.FindIndex(x => x.Id == id);

```

```

        if (index < 0) return NotFound("Студент не найден.");

        _students.RemoveAt(index);
        return NoContent();
    }

    private static List<Student> _students = new();
}

```

Листинг 5. Файл StudentModel.cs:

```

namespace Lab9.Api.Students.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Group { get; set; }
        public string Specialization { get; set; }
    }
}

```

Листинг 6. Файл StudentApiOptions.cs:

```

namespace Lab9.Api.Students.Options
{
    public class StudentApiOptions
    {
        public int MaxCountStudents { get; init; }
        public bool AllowAdd { get; init; }
        public bool AllowDelete { get; init; }
        public bool AllowUpdate { get; init; }
        public string Code { get; init; }
    }
}

```

Листинг 7. Файл launchSettings.json:

```

{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "applicationUrl": "http://localhost:5176",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Production",
        "StudentApiSettings__Code": "12345"
      }
    }
  }
}

```

Листинг 8. Файл appsettings.json:

```

{
  "StudentApiSettings": {
    "MaxCountStudents": 10,
    "AllowAdd": true,

```

```

        "AllowDelete": true,
        "AllowUpdate": true,
        "Code": "FROM <ENV>"
    },
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*"
}

```

Листинг 9. Файл appsettings.Development.json:

```

{
  "StudentApiSettings": {
    "MaxCountStudents": 5,
    "AllowAdd": false,
    "AllowDelete": false,
    "AllowUpdate": false
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

```

Листинг 10. Файл appsettings.Production.json:

```

{
  "StudentApiSettings": {
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

```

Листинг 11. Файл appsettings.Testing.json:

```

{
  "StudentApiSettings": {
    "MaxCountStudents": 10000
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

```